Simulation of Simple Harmonic Motion using RK4 in MATLAB

October 13, 2025

Abstract

This report details the numerical simulation of a mass-spring-damper system, a canonical example of Simple Harmonic Motion (SHM). The dynamics are modeled by a second-order ordinary differential equation (ODE). A fixed-step fourth-order Runge-Kutta (RK4) integrator is implemented in MATLAB to solve the system. The analysis covers free, damped (underdamped, critically damped, overdamped), and driven oscillations. System behavior is visualized and analyzed through time-domain displacement plots and phase-space portraits.

1 Governing Equations

The general equation for a driven, damped harmonic oscillator is:

$$m\ddot{x} + b\dot{x} + kx = F_0 \cos(\omega_F t) \tag{1}$$

where m is the mass, b is the damping coefficient, k is the spring constant, and $F_0 \cos(\omega_F t)$ is the external driving force.

To solve this numerically, we convert the second-order ODE into a system of two first-order ODEs. Let the state vector be $\mathbf{y} = [x, v]^T$, where $v = \dot{x}$. The system becomes:

$$\frac{d\mathbf{y}}{dt} = \begin{pmatrix} \dot{x} \\ \dot{v} \end{pmatrix} = \begin{pmatrix} v \\ \frac{1}{m}\left(-bv - kx + F_0 \cos(\omega_F t)\right) \end{pmatrix} \tag{2}$$

2 Task 1 — Basic Numeric Solver & Plotting

The first task is to implement the RK4 solver for a specific case of a damped, undriven oscillator and visualize its dynamics. The system is undriven, so we must set $F_0 = 0.0$.

2.1 Parameters

- Mass, $m = 1.0$ kg
- Spring constant, $k = 10.0$ N/m
- Damping coefficient, $b = 0.4$ Ns/m
- Driving force amplitude, $F_0 = 0.0$ N
- Driving frequency, $\omega_F = 0.0$ rad/s
- Initial conditions: $x(0) = 1.0$ m, $v(0) = 0.0$ m/s

2.2 MATLAB Code for Task 1

```
% MATLAB SCRIPT FOR SHM SIMULATION - TASK 1
clear; close all; clc;

%% --- Parameters & Setup ---
m = 1.0;          % Mass (kg)
k = 10.0;         % Spring constant (N/m)
b = 0.4;          % Damping coefficient (Ns/m)
F0 = 0.0;         % Driving force amplitude (N) - Set to 0 for undriven case
wF = 0.0;         % Driving frequency (rad/s)

% Simulation time
t_start = 0;
t_end = 20;
dt = 0.01;        % Time step
t = t_start:dt:t_end; % Time vector
N = length(t);    % Number of steps

% Initial conditions: state vector y = [x; v]
y0 = [1.0; 0.0];  % Start from x=1 with zero initial velocity

%% Define the System of ODEs
% y(1) = x, y(2) = v
% dy/dt = [v; (-b*v - k*x + F0*cos(wF*t))/m]
f = @(t, y) [y(2); (-b*y(2) - k*y(1) + F0*cos(wF*t))/m];

%% RK4 Integration
y = zeros(2, N);
y(:, 1) = y0;
for i = 1:(N-1)
    k1 = dt * f(t(i), y(:, i));
    k2 = dt * f(t(i) + 0.5*dt, y(:, i) + 0.5*k1);
    k3 = dt * f(t(i) + 0.5*dt, y(:, i) + 0.5*k2);
    k4 = dt * f(t(i) + dt, y(:, i) + k3);
    y(:, i+1) = y(:, i) + (k1 + 2*k2 + 2*k3 + k4)/6;
end
x = y(1, :);
v = y(2, :);

%% Plotting for Task 1
figure('Name', 'Task 1: Damped Oscillation');
% Displacement vs. Time
subplot(1, 2, 1);
plot(t, x, 'b-', 'LineWidth', 1.5);
title('Displacement vs. Time');
xlabel('Time (s)');
ylabel('Displacement x(t)');
grid on;
% Phase Space
subplot(1, 2, 2);
```

```
plot(x, v, 'r-', 'LineWidth', 1.5);
hold on;
plot(x(1), v(1), 'go', 'MarkerFaceColor', 'g', 'MarkerSize', 8); % Start
plot(0, 0, 'kx', 'MarkerSize', 10, 'LineWidth', 2); % Equilibrium
title('Phase Space Plot');
xlabel('Displacement x');
ylabel('Velocity v');
axis equal;
grid on;
legend('Trajectory', 'Start', 'Equilibrium', 'Location', 'northeast');
```

Listing 1: Standalone MATLAB code for Task 1.

2.3 Results and Analysis

- **Displacement vs. Time Plot:** The plot of $x(t)$ shows a sinusoidal oscillation whose amplitude decays exponentially. This is the classic signature of an underdamped system where energy is gradually dissipated by the damping force.
- **Phase-Space Plot:** The plot of velocity v versus displacement x reveals a trajectory that begins at $(1, 0)$ and spirals inward, asymptotically approaching the equilibrium point at the origin $(0, 0)$. This point acts as a **stable spiral attractor**, confirming the underdamped nature of the system.

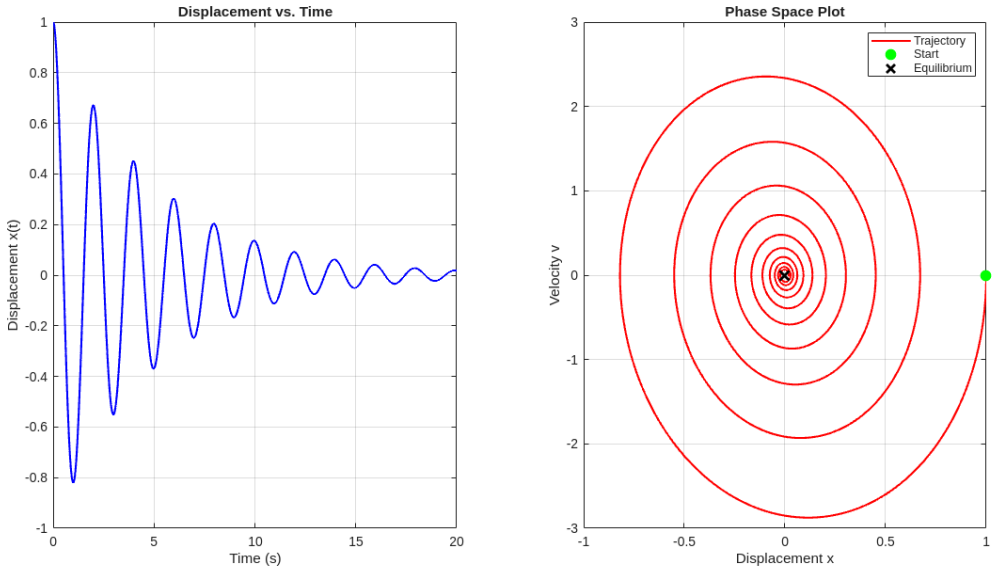


Figure 1: Task 1: Displacement vs. Time (Left) and Phase-Space Portrait (Right) for a damped oscillator.

3 Task 2 — Comparative Study of Linear Cases

This task compares the behavior of the oscillator under different damping and driving conditions by modifying the system parameters and re-running the simulation.

3.1 MATLAB Code for Task 2

This code uses a helper function to avoid repetition and generates plots for all required cases. The "Driven Damped" case uses $F_0 = 0.5$.

```
% MATLAB SCRIPT FOR SHM SIMULATION - TASK 2
clear; close all; clc;

%% --- Common Parameters & Setup ---
m = 1.0;          % Mass (kg)
k = 10.0;         % Spring constant (N/m)
w0 = sqrt(k/m);   % Natural frequency (rad/s)

% Simulation time
t_start = 0;
t_end = 40;       % Extended time for driven case to show steady state
dt = 0.01;        % Time step
t = t_start:dt:t_end; % Time vector

% Initial conditions: [x(0); v(0)]
y0 = [1.0; 0.0];  % Start from x=1 with zero initial velocity

%% --- Run All Cases ---
% Case 1: Free SHM (b=0, F0=0)
run_and_plot_case(t, y0, m, k, dt, 'Free SHM', 0, 0, 0);

% Case 2: Underdamped (b=0.4, F0=0) - using value from prompt
run_and_plot_case(t, y0, m, k, dt, 'Underdamped', 0.4, 0, 0);

% Case 3: Critically Damped (b=2*sqrt(k*m), F0=0)
b_crit = 2*sqrt(k*m);
run_and_plot_case(t, y0, m, k, dt, 'Critically Damped', b_crit, 0, 0);

% Case 4: Overdamped (b > b_crit, F0=0)
run_and_plot_case(t, y0, m, k, dt, 'Overdamped', b_crit * 2, 0, 0);

% Case 5: Driven Damped at Resonance (b=0.4, F0=0.5, wF=w0)
run_and_plot_case(t, y0, m, k, dt, 'Driven Damped (F0=0.5)', 0.4, 0.5, w0);


%% --- Helper Function Definition ---
function run_and_plot_case(t, y0, m, k, dt, case_title, b, F0, wF)
    fprintf('Running case: %s\n', case_title);
    N = length(t);

    % Define the ODE system for the current case
    f = @(t, y) [y(2); (-b*y(2) - k*y(1) + F0*cos(wF*t))/m];

    % RK4 Integration
    y = zeros(2, N);
    y(:, 1) = y0;
    for i = 1:(N-1)
        k1 = dt * f(t(i), y(:, i));
        k2 = dt * f(t(i) + 0.5*dt, y(:, i) + 0.5*k1);
        k3 = dt * f(t(i) + 0.5*dt, y(:, i) + 0.5*k2);
        k4 = dt * f(t(i) + dt, y(:, i) + k3);
        y(:, i+1) = y(:, i) + (k1 + 2*k2 + 2*k3 + k4)/6;
    end
    x = y(1, :);
    v = y(2, :);

    % Calculate Energy E(t) = 0.5*m*v^2 + 0.5*k*x^2
    energy = 0.5*m*v.^2 + 0.5*k*x.^2;

    % Plotting
    figure('Name', ['Task 2: ' case_title]);
    sgtitle(case_title, 'FontSize', 14, 'FontWeight', 'bold');
```

```
63
64      % Displacement vs. Time
65      subplot(1, 3, 1);
66      plot(t, x, 'LineWidth', 1.5);
67      title('Displacement x(t)');
68      xlabel('Time (s)'); ylabel('Displacement x(t)'); grid on;
69
70      % Phase Space
71      subplot(1, 3, 2);
72      plot(x, v, 'LineWidth', 1.5); hold on;
73      plot(x(1), v(1), 'go', 'MarkerFaceColor', 'g', 'MarkerSize', 8);
74      plot(0, 0, 'kx', 'MarkerSize', 10, 'LineWidth', 2);
75      title('Phase Portrait (v vs. x)');
76      xlabel('Displacement x'); ylabel('Velocity v'); axis equal; grid on;
77
78      % Energy vs. Time
79      subplot(1, 3, 3);
80      plot(t, energy, 'LineWidth', 1.5);
81      title('Total Energy E(t)');
82      xlabel('Time (s)'); ylabel('Energy (J)'); grid on;
83  end
```

Listing 2: Standalone MATLAB code for Task 2.

3.2 Results and Analysis

- **Free SHM:** The $x(t)$ plot is a constant-amplitude sine wave. The phase portrait is a **closed ellipse**, indicating conserved energy, which is confirmed by the flat energy plot. The origin is a stable center.
- **Underdamped:** The $x(t)$ plot shows decaying oscillations. The phase portrait is an **inward spiral** to the origin, and the energy plot shows an exponential decay.
- **Critically Damped:** The system returns to equilibrium in the fastest possible time without overshooting. The phase portrait shows a **direct path** to the origin.
- **Overdamped:** The system returns to equilibrium **slowly and monotonically** (no oscillations). The phase portrait is also a direct path to the origin but represents a slower convergence than the critically damped case.
- **Driven Damped:** Initially, the motion is a superposition of the natural damped response and the driving force (transient state). After the transients decay, the system settles into a steady-state oscillation. The phase portrait converges to a stable **limit cycle** instead of a point, representing the balance between energy input by the driving force and energy dissipated by damping.

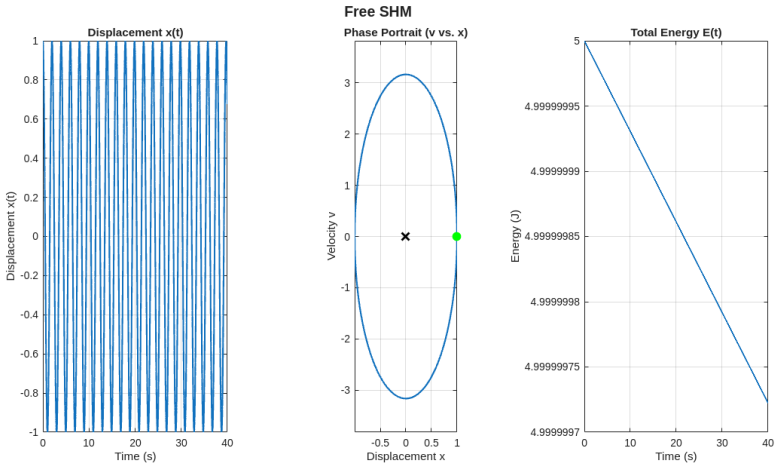


Figure 2: Free SHM

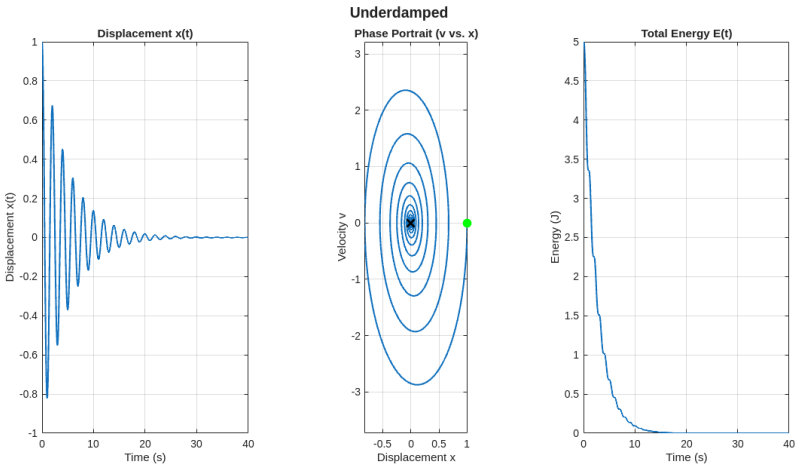


Figure 3: Underdamped

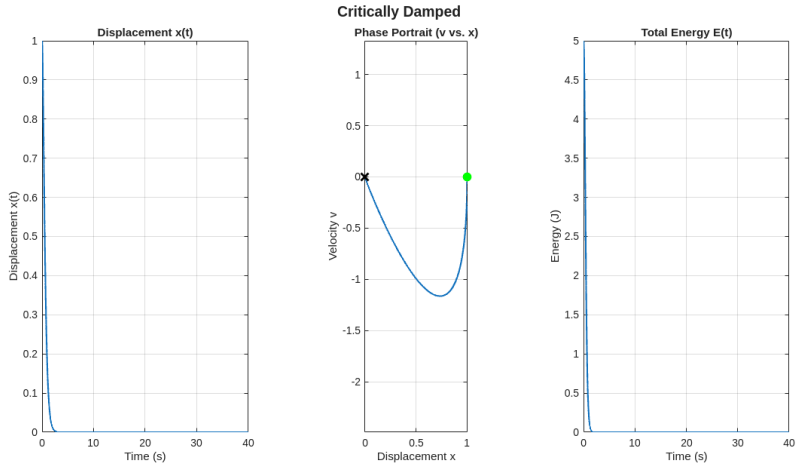


Figure 4: Critically Damped

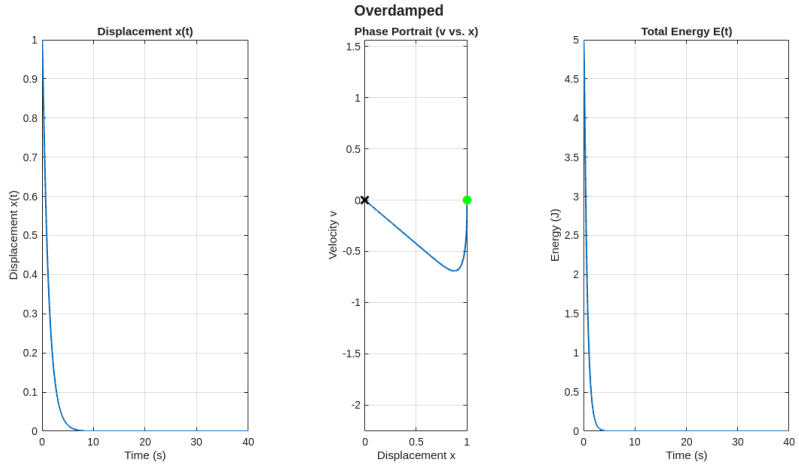


Figure 5: Overdamped

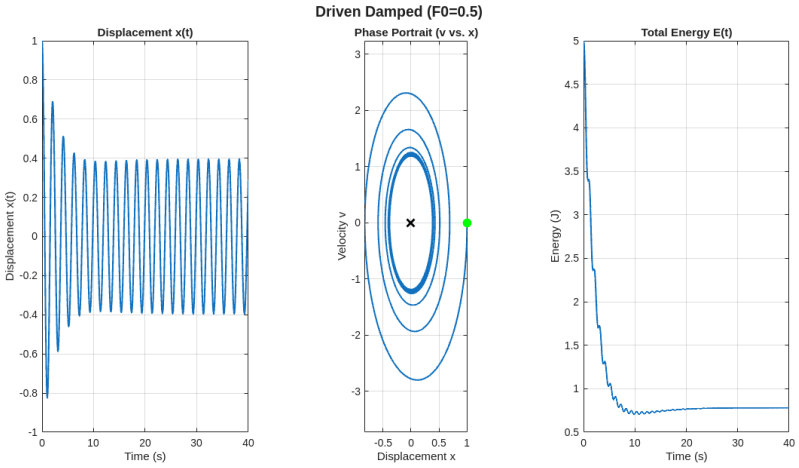


Figure 6: Driven Damped